

COMMISSIONING PLATFORM

Interface Contracts, Schemas,
and Architecture Specification

Companion to Technical Specification v5 | April 2026

1. Backend Architecture

1.1 Process Model

The backend is a single Python process using asyncio. All protocol workers, the test engine, the orchestrator, and the API server run as async tasks within one event loop. No threads. No multiprocessing. Asyncio handles concurrency because all work is I/O-bound (network polling, DB writes, API calls).

Process structure:

- main.py: entry point. Starts the asyncio event loop. Launches all tasks.
- One FastAPI server task (serves frontend, Module 18-20)
- One discovery task (Module 11, runs once at startup or on demand)
- One worker task per protocol type polling all devices of that protocol (Modules 6-9)
- One orchestrator task managing test execution queue (Module 13)
- One attestation task consuming a queue of records to hash and write (Module 5)

! Single process, single event loop. If one task blocks (e.g. synchronous file I/O), everything stalls. All I/O must be async. Use aiofiles for file operations, aiohttp for HTTP clients, asyncio.to_thread() for CPU-bound work like PDF parsing.

1.2 Docker Compose Stack

Seven containers. All on a single Docker network called 'cx-net'.

Container	Image	Ports	Volumes	Env
netbox	netboxcommunity/netbox:latest	8000:8080	netbox-media:/opt/netbox/media	See NetBox docs
netbox-postgres	postgres:16	none (internal)	netbox-pgdata:/var/lib/postgresql/data	POSTGRES_DB=netbox
netbox-redis	redis:7-alpine	none (internal)	none	none
influxdb	influxdb:2.7	8086:8086	influx-data:/var/lib/influxdb2	DOCKER_INFLUXDB_INIT_*
minio	minio/minio:latest	9000:9000, 9001:9001	minio-data:/data	MINIO_ROOT_USER/ PASSWORD
platform	python:3.12-slim (custom Dockerfile)	8080:8080	./config:/app/config	See 1.4 below
simulator	python:3.12-slim (dev only)	5020:502	none	SIM_PROFILE=ups_transfer

Docker network: all containers on 'cx-net'. Platform container reaches NetBox at <http://netbox:8080>, InfluxDB at <http://influxdb:8086>, MinIO at <http://minio:9000>. Facility devices

are reached via the host network (`network_mode: host` on platform container) or via port mapping to the facility subnet.

⚠ Platform container must be on the facility network to reach devices. Either use `network_mode: host`, or add the facility subnet as a second network on the container. Docker bridge networking will NOT reach devices on the facility LAN.

1.3 Health Checks

Component	Health Check	Interval	Unhealthy Action
NetBox	GET /api/status/	30s	Platform logs error, continues with cached data
InfluxDB	GET /health	30s	Workers buffer measurements in memory, flush on recovery
MinIO	mc admin info minio	30s	Attestation queues to local WAL, flushes on recovery
Modbus worker	Tracks consecutive poll failures per device	per-poll	3 consecutive failures = device marked UNREACHABLE
BACnet worker	Same pattern	per-poll	Same pattern
SNMP worker	Same pattern	per-poll	Same pattern
Attestation task	Queue depth monitored	5s	If queue > 10,000 records, log warning, backpressure workers
Orchestrator	Running test count vs max_concurrent	per-dispatch	If at max, queue test, don't drop

1.4 Configuration

Single YAML file at /app/config/platform.yml. All settings in one place. No env vars except Docker infrastructure (passwords).

Key	Type	Description
facility.name	string	Facility identifier for attestation records
facility.scan_subnets[]	string[]	CIDR ranges to scan during discovery (e.g. [10.0.0.0/24, 10.0.1.0/24])
netbox.url	string	http://netbox:8080
netbox.token	string	NetBox API token
influxdb.url	string	http://influxdb:8086
influxdb.token	string	InfluxDB API token
influxdb.org	string	InfluxDB organization
influxdb.bucket	string	Bucket name (default: commissioning)
minio.endpoint	string	minio:9000
minio.access_key	string	MinIO access key
minio.secret_key	string	MinIO secret key
minio.bucket	string	Bucket name (default: attestation)
minio.retention_days	int	WORM retention period (default: 2555 = 7 years)

workers.max_concurrent_polls	int	Max simultaneous device polls per protocol (default: 50)
orchestrator.max_concurrent_tests	int	Max active tests running simultaneously (default: 5)
api.port	int	FastAPI listen port (default: 8080)
api.cors_origins[]	string[]	Allowed CORS origins for frontend
ntp.server	string	NTP server for platform clock (default: pool.ntp.org)
logging.level	string	DEBUG INFO WARNING ERROR
logging.format	string	json (structured logging, default)
external_apis.genetec.url	string	Genetec Security Center API URL
external_apis.genetec.token	string	API token
external_apis.servicenow.url	string	ServiceNow instance URL
external_apis.servicenow.token	string	API token

2. Inter-Module Communication

Modules communicate via three mechanisms only. No message queues. No external brokers. Pure Python.

2.1 Direct Function Calls

Modules that run within the same async task call each other's functions directly. The Modbus worker (Module 6) calls the Modbus poller (Module 3), then calls the InfluxDB writer (Module 4), then calls the attestation engine (Module 5). These are just async function calls within the same task.

2.2 `asyncio.Queue`

Cross-task communication uses `asyncio.Queue`. Three queues exist:

Queue	Producer	Consumer	Item Type
attestation_queue	All workers (Modules 6-9), test engine (Module 10), REST connectors (Module 12), checklist (via API)	Attestation task (Module 5)	AttestationRecord dataclass
test_queue	API server (when user clicks Run Test) or orchestrator batch	Orchestrator (Module 13)	TestRequest dataclass
event_queue	All modules on state changes	API server (for SSE to frontend)	Event dataclass (for real-time dashboard updates)

2.3 Shared State

Read-only shared state via a single `FacilityState` object initialized at startup, holding the platform config and NetBox client instance. Workers read from it. Nobody writes to it after startup except the discovery scanner which updates the device list.

Mutable shared state is avoided. Each worker tracks its own per-device state (last poll time, consecutive failures, current value cache) in local dicts. The orchestrator tracks test execution state in its own dict. No shared mutable state between tasks.

3. Data Schemas

3.1 Python Dataclasses (shared types between all modules)

DeviceInfo (from Module 2, consumed by Modules 3-13):

Field	Type	Source
device_id	str	NetBox device ID
name	str	NetBox device name
primary_ip	str	NetBox primary IP (or connection.host from Config Context)
device_type_slug	str	NetBox device type slug
config_context	dict	Full Config Context JSON from NetBox
protocol	str	Extracted from config_context['protocol']
site	str	NetBox site name
rack	str	NetBox rack name
position	int	NetBox rack position

PollResult (from Modules 3, 7, 8, 9, consumed by Modules 4, 5, 6, 10):

Field	Type	Description
device_id	str	NetBox device ID
timestamp_ns	int	Platform NTP time in nanoseconds
measurements	dict[str, float]	Register name → decoded scaled value
raw_bytes	dict[str, str]	Register name → hex string of raw bytes from device
protocol	str	modbus_tcp bacnet_ip snmp nvml
source_ip	str	IP address polled
success	bool	True if all registers read successfully
errors	dict[str, str]	Register name → error message for failed reads

AttestationRecord (from all producers, consumed by Module 5):

Field	Type	Description
timestamp_ns	int	Platform NTP nanoseconds
device_id	str	NetBox device ID
measurement	str	Register name or metric name
value	float	Decoded scaled value
raw_bytes	str	Hex string

protocol	str	Protocol used
source_ip	str	Device IP
worker_id	str	Unique worker instance ID (hostname-pid-protocol)
test_id	str None	Active test execution ID, or None for regular polling
facility	str	From platform config
nonce	str	worker_id + monotonic sequence number

AttestedRecord (output of Module 5, stored in MinIO):

Field	Type	Description
...all AttestationRecord fields		Inherited
hash	str	SHA-256 hex of canonical JSON of (timestamp_ns, device_id, measurement, value, raw_bytes, previous_hash)
previous_hash	str	Hash of prior record in chain
chain_id	str	Chain identifier: facility_id (one chain per facility)
sequence	int	Monotonic sequence number within chain

TestRequest (from API/orchestrator, consumed by Module 10):

Field	Type	Description
test_id	str	UUID generated at request time
device_id	str	NetBox device ID to test
test_name	str	Name from active_tests[] in Config Context
requested_by	str	Username or 'orchestrator'
requested_at	str	ISO timestamp
priority	int	1 (highest) to 5 (lowest). Default 3.

TestResult (output of Module 10):

Field	Type	Description
test_id	str	From TestRequest
device_id	str	Device tested
test_name	str	Test name
status	str	passed failed aborted restore_failure precondition_failed manual_pending
started_at	str	ISO timestamp
completed_at	str	ISO timestamp
duration_seconds	float	Total test duration
precondition_results	list[dict]	[[register, expected, actual, passed]]
manual_confirmations	list[dict]	[[prompt, confirmed_by, confirmed_at]]
acceptance_results	list[dict]	[[register, metric, expected, actual, passed]]
abort_triggered	bool	Whether an abort condition fired
abort_reason	str None	Which abort condition triggered
restore_success	bool	Whether restore completed successfully
evidence_hashes	list[str]	All attestation hashes generated during this test
contact_wear_before	float None	For breaker tests: contact wear % pre-test
contact_wear_after	float None	For breaker tests: contact wear % post-test

PunchListItem (output of Module 14):

Field	Type	Description
id	str	UUID
severity	str	critical major minor info
category	str	identity firmware network power cooling security safety test_failure physical

device_id	str	NetBox device ID
device_name	str	Human-readable device name
site	str	Facility/site name
rack	str	Rack location
expected	str	Value from design spec
actual	str	Value from discovery or test
source	str	discovery passive_read active_test reconciliation physical_checklist
evidence_hash	str None	SHA-256 of attestation record
test_id	str None	If from active test
remediation	str	Suggested corrective action
status	str	open acknowledged resolved deferred
resolved_by	str None	Who resolved it
resolved_at	str None	When resolved

Event (for real-time dashboard updates via SSE):

Field	Type	Description
event_type	str	device_discovered poll_result test_started test_completed test_aborted punch_item worker_health manual_confirmation_needed
timestamp	str	ISO timestamp
data	dict	Event-specific payload (varies by type)

3.2 InfluxDB Schema

One bucket: 'commissioning'. All measurements use the same tag/field pattern.

Element	Key	Type	Description
Measurement name	poll	n/a	All regular polling data uses this measurement
Measurement name	test	n/a	All data captured during active tests uses this
Tag	device_id	string	NetBox device ID
Tag	device_name	string	Human-readable name
Tag	protocol	string	modbus_tcp bacnet_ip snmp nvml
Tag	site	string	Facility name
Tag	rack	string	Rack identifier
Tag	test_id	string	Only on 'test' measurement. Links to TestResult.
Field	(register_name)	float	One field per register. Field name = register name from Config Context.

Example InfluxDB write (line protocol):

```
poll,device_id=dev123,device_name=CM2000-A3,protocol=modbus_tcp,site=DC1,rack=A3
current_phase_a=450.0,voltage_ab=480.2,frequency_hz=60.01 1712847600000000000
```

Retention policies:

- Raw data: 90 days (full resolution for active commissioning)
- Downsampled (1-minute averages): 2 years (post-commissioning reference)
- Continuous queries handle downsampling automatically

3.3 MinIO Structure

One bucket: 'attestation'. Object Lock enabled in compliance mode at bucket creation (cannot be disabled afterward).

Key format: {facility}/{date}/{chain_sequence}.json

Example: DC1-Ashburn/2026-04-12/000001423.json

Each object is a single AttestedRecord serialized as JSON. The retention period is set per-object at write time from the platform config (minio.retention_days). Objects cannot be deleted or overwritten until the retention period expires.

WAL (Write-Ahead Log) for WORM failures:

- Location: /app/data/wal/ on the platform container
- Format: one JSON file per failed batch, named {timestamp_ns}.json
- Recovery: attestation task checks WAL directory every 60 seconds, replays any files found, deletes WAL file on successful WORM write
- The WAL volume is a Docker volume (cx-wal) that persists across container restarts

4. API Contracts (FastAPI)

All endpoints prefixed with /api/v1. JSON request/response. Authentication via API key in X-API-Key header (configured in platform.yml). Frontend served as static files from /app/frontend/build at root /.

4.1 Discovery

Method	Path	Request	Response	Description
POST	/discovery/scan	{subnets: ['10.0.0.0/24']}	{scan_id: str, status: 'running'}	Start discovery scan. Uses config subnets if none provided.
GET	/discovery/status/{scan_id}	none	{scan_id, status, devices_found: int, devices_classified: int, devices_unmatched: int, errors: []}	Poll scan progress.
GET	/discovery/results/{scan_id}	none	{devices: [DeviceInfo]}	Get discovered devices.

4.2 Devices

Method	Path	Request	Response	Description
GET	/devices	? protocol=modbus_tcp&site=DC1	{devices: [DeviceInfo]}	List devices from NetBox with filters.
GET	/devices/{device_id}	none	{device: DeviceInfo, last_poll: PollResult, tests_available: [str], tests_run: [TestResult]}	Single device with latest data and test history.
GET	/devices/{device_id}/history	? register=current_phase_a &from=2026-04-01&to=2026-04-12	{points: [{timestamp, value}]}	InfluxDB query for register history.

4.3 Tests

Method	Path	Request	Response	Description
GET	/tests/available/{device_id}	none	{tests: [{name, description, preconditions}]}	List available active tests for device from Config

				Context.
POST	/tests/run	{device_id, test_name}	{test_id, status: 'queued'}	Queue a test. Orchestrator dispatches.
POST	/tests/confirm/{test_id}	{confirmed_by: 'engineer_name'}	{status: 'confirmed'}	Satisfy manual_confirmation for a pending test.
GET	/tests/status/{test_id}	none	{TestResult}	Get test result with all acceptance criteria, evidence hashes.
GET	/tests/live/{test_id}	SSE stream	Event stream: {register_name, value, timestamp} per monitor_interval	Real-time register values during active test.
POST	/tests/abort/{test_id}	none	{status: 'aborting'}	Force abort. Executes restore immediately.
GET	/tests/history	? device_id=X&status=failed	{tests: [TestResult()]}	Query test history with filters.

4.4 Punch List

Method	Path	Request	Response	Description
GET	/punchlist	? severity=critical&category=power&status=open	{items: [PunchListItem]}	Filtered punch list.
PATCH	/punchlist/{id}	{status: 'resolved', resolved_by: 'name'}	{item: PunchListItem}	Update punch list item status.
GET	/punchlist/summary	none	{total, by_severity: {critical: N, major: N, ...}, by_category: {...}, by_status: {...}}	Dashboard summary counts.

4.5 Attestation

Method	Path	Request	Response	Description
GET	/attestation/{hash}	none	{record: AttestedRecord}	Retrieve single attested record by hash.
GET	/attestation/chain	? from_sequence=100 &count=50	{records: [AttestedRecord[]], chain_valid: bool}	Walk the chain. Returns chain validity check.

GET	/attestation/verify/{hash}	none	{valid: bool, chain_position: int, chain_length: int, breaks: []}	Verify a specific record's chain integrity.
GET	/attestation/export/{test_id}	none	application/json file download	Export all attested records for a test as signed bundle.
GET	/attestation/certificate	none	{facility, chain_length, chain_valid, first_record, last_record, hash_algorithm}	Facility-wide attestation certificate.

4.6 Physical Checklist

Method	Path	Request	Response	Description
GET	/checklist/{device_id}	none	{items: [{id, description, category, required_photo: bool, completed: bool}]}	Get checklist for device based on device type.
POST	/checklist/{device_id}/{item_id}	multipart: {completed: bool, notes: str, photo: file}	{item: updated, attestation_hash: str}	Submit checklist item with optional photo. Attested.
GET	/checklist/summary	none	{total_items, completed, remaining, by_device: [...]}	Overall physical verification progress.

4.7 Reconciliation and Reports

Method	Path	Request	Response	Description
POST	/reconciliation/run	none	{job_id, status: 'running'}	Trigger reconciliation (design vs actual).
GET	/reconciliation/status/{job_id}	none	{status, punch_items_generated: int}	Poll progress.
POST	/reports/generate	{include_sections: ['discovery', 'tests', 'punchlist', 'attestation']}	{report_id, status: 'generating'}	Generate PDF report.
GET	/reports/{report_id}	none	application/pdf file download	Download generated report.

4.8 BIM Import

Method	Path	Request	Response	Description
POST	/bim/import	multipart: {file: .ifc}	{job_id, status: 'parsing'}	Upload IFC file for import.
GET	/bim/status/{job_id}	none	{status, devices_parsed, devices_created, errors}	Poll import progress.
GET	/bim/preview/{job_id}	none	{devices: [{name, type, location, connections}]}	Preview parsed devices before committing to NetBox.
POST	/bim/commit/{job_id}	none	{devices_committed: int}	Commit parsed devices to NetBox.

4.9 Config Context Generation

Method	Path	Request	Response	Description
POST	/config/generate	multipart: {pdf: file, device_type_slug: str}	{job_id, status: 'extracting'}	Send manufacturer PDF, get Config Context.
GET	/config/result/{job_id}	none	{config_context: dict, validation_errors: [], warnings: []}	Get generated Config Context with validation results.
POST	/config/commit/{job_id}	none	{status: 'committed'}	Load validated Config Context into NetBox.

4.10 System

Method	Path	Request	Response	Description
GET	/system/health	none	{netbox: ok error, influxdb: ok error, minio: ok error, workers: {modbus: {devices: N, errors: N}, ...}}	Overall health.
GET	/system/events	SSE stream	Event stream	Server-Sent Events for real-time dashboard updates.

GET	/system/config	none	{facility_name, scan_subnets, ...} (passwords redacted)	Current platform config.
-----	----------------	------	--	--------------------------

5. Algorithms

5.1 Power Dependency Graph

The orchestrator must never schedule tests on devices that share a power chain simultaneously. The power dependency graph is built from NetBox's power port, power outlet, and power feed objects.

Algorithm:

- 1. Query NetBox API for all power connections (power ports connected to power outlets).
- 2. Build a directed acyclic graph (DAG): edge from power_source → powered_device for each connection.
- 3. For each device with a pending test, walk UP the DAG to find all ancestor power sources.
- 4. Two tests CONFLICT if they share any ancestor in the DAG.
- 5. Conflicting tests are serialized. Non-conflicting tests run in parallel up to max_concurrent_tests.

Data structure: dict[device_id, set[ancestor_device_ids]]. Precomputed once from NetBox at startup and after any discovery update.

Scheduling algorithm:

- 1. Sort test queue by priority (1=highest), then by request time.
- 2. For each queued test, check if any currently running test conflicts (shared ancestors).
- 3. If no conflict and under max_concurrent_tests, dispatch. Else keep in queue.
- 4. When a test completes, re-evaluate queue.

⚠ If power connections are not modeled in NetBox, the orchestrator has NO conflict data and will allow all tests to run concurrently. The BIM import (Module 16) must create power connections in NetBox, not just devices.

5.2 Reconciliation Algorithm

Compares design state (NetBox objects created by BIM import) against actual state (discovered devices + test results).

For each device in the design (NetBox):

- 1. IDENTITY CHECK: Was it discovered? If not → PunchListItem(severity=critical, category=identity, expected=device_type, actual='NOT FOUND').

- 2. MODEL CHECK: Does discovered model match design model? Compare device_type_slug. Mismatch → PunchListItem(severity=major, category=identity).
- 3. FIRMWARE CHECK: Does firmware_version match Config Context's firmware_version? Mismatch → PunchListItem(severity=minor, category=firmware).
- 4. LOCATION CHECK: Is the device in the correct rack and position? Compare NetBox rack and position. Mismatch → PunchListItem(severity=major, category=identity).
- 5. NETWORK CHECK: Do LLDP neighbors match the cable schedule in NetBox? Compare each interface's connected endpoint. Mismatch → PunchListItem(severity=major, category=network).
- 6. POWER CHECK: Is the device on the correct power source? Compare power port connection in NetBox. Mismatch → PunchListItem(severity=major, category=power).
- 7. TEST CHECK: Did all active tests pass? For each test in TestResult history, if status != passed → PunchListItem(severity based on test type: safety/power=critical, other=major, category=test_failure).

For each discovered device NOT in the design:

- PunchListItem(severity=info, category=identity, expected='NOT IN DESIGN', actual=discovered_device_type). Unexpected device.

Severity determination:

Condition	Severity
Device in design not found (not powered on, not installed)	critical
Active test failed with safety implications (UPS, ATS, breaker)	critical
Wrong device model installed	major
Wrong rack position	major
Wrong network connection	major
Wrong power source	major
Active test failed (non-safety: cooling, environmental)	major
Firmware mismatch	minor
IP address mismatch	minor
DNS/NTP configuration mismatch	minor
Unexpected device found (not in design)	info

5.3 Active Test Execution Algorithm

Module 10 executes each test as a state machine:

States: INIT → PRECONDITION_CHECK → MANUAL_WAIT (if needed) → COMMANDING → MONITORING → EVALUATING → RESTORING → RESTORE_VERIFY → COMPLETE

- INIT: Load test definition from Config Context. Set test_id. Log start.
- PRECONDITION_CHECK: Read each precondition register. Evaluate each condition. If ANY fails → status=precondition_failed, log which precondition, skip to COMPLETE. All pass → continue.
- MANUAL_WAIT: If manual_confirmation[] is non-empty, set status>manual_pending. Push event to SSE. Wait for POST /tests/confirm/{test_id}. Timeout after 30 minutes → status=aborted.
- COMMANDING: Read identity_register (if defined) and verify device identity. Mismatch → abort. Write command to device. Log write. If write fails → status=aborted.
- MONITORING: Start monitor loop. Every monitor_interval_ms, read all monitor[] registers. For each read, create AttestationRecord with test_id set. Push to attestation_queue. Check abort_conditions[] against every read. If ANY abort triggers → immediately jump to RESTORING with abort_triggered=true.
- EVALUATING: After monitor_duration_seconds, evaluate all acceptance[] criteria against the collected data. For time_to_value: find the first timestamp where the register crossed the threshold. For max/min/avg: compute across all readings. For delta: compute first reading minus last reading. For settled_value: average of last 10% of readings. For rate_of_change: linear regression slope across all readings.
- RESTORING: Write restore command. Log write.
- RESTORE_VERIFY: Read restore_verify[] registers. Poll every 1 second for up to restore_timeout_seconds. If all verify conditions met → restore_success=true. If timeout → restore_success=false, status=restore_failure.
- COMPLETE: Build TestResult. Push punch list items for any failed acceptance criteria. Push event to SSE.

⚠ The MONITORING loop must enforce a watchdog. If 3 consecutive poll attempts fail (device unreachable), immediately jump to RESTORING with abort_triggered=true and abort_reason='device_unreachable'. Execute restore BLINDLY (fire-and-forget the write, don't wait for read confirmation) because you can't read from the device. Log ABORTED with reason.

5.4 Attestation Chain Algorithm

Module 5 maintains one chain per facility. The chain is a strictly ordered sequence. No branching.

- 1. Attestation task consumes from `attestation_queue`.
- 2. For each record: read `previous_hash` from memory (last hash in the chain).
- 3. Build canonical JSON string: sort keys alphabetically, no whitespace. Fields: `timestamp_ns`, `device_id`, `measurement`, `value`, `raw_bytes`, `previous_hash`.
- 4. Compute SHA-256 of the canonical JSON string.
- 5. Set `record.hash` = computed hash, `record.previous_hash` = previous hash, `record.sequence` = `last_sequence` + 1.
- 6. Serialize full `AttestedRecord` as JSON.
- 7. Write to MinIO with Object Lock (retention = `minio.retention_days`). Key = `{facility}/{date}/{sequence}.json`.
- 8. On success: update in-memory `previous_hash` and `last_sequence`. Also write to InfluxDB for queryability.
- 9. On MinIO failure: write to WAL at `/app/data/wal/{timestamp_ns}.json`. Do NOT update `previous_hash` (chain pauses). Log error. Retry from WAL on next 60-second WAL sweep.

Chain verification (Module 20 / API): Walk from any record backward. For each record, recompute the hash from the canonical JSON fields and `previous_hash`. If recomputed hash != stored hash, the chain is broken at that point.

⚠ The chain must be single-threaded. Only one goroutine/task may append to the chain at a time. The `attestation_queue` serializes all records into one consumer task. If multiple tasks write to the chain concurrently, sequence numbers and `previous_hash` will collide.

5.5 Discovery Classification Algorithm

Module 11 discovers devices and attempts to match them to NetBox device types.

- 1. For each IP in the scan range, probe in parallel: SNMP GET sysDescr (port 161), Modbus TCP device ID function code 43 (port 502), BACnet/IP Who-Is (port 47808), Redfish GET /redfish/v1 (port 443).
- 2. Collect all responses. A single IP may respond on multiple protocols.
- 3. For each response, extract the device identity string (sysDescr for SNMP, device_id for Modbus, object_name for BACnet, Model for Redfish).
- 4. Search NetBox device types for a match. Match algorithm: case-insensitive substring match of identity string against device type manufacturer + model fields. If multiple matches, prefer exact match over substring.
- 5. If matched: create or update NetBox device object with discovered IP, device type, and protocol. Attach Config Context from the matched device type.
- 6. If unmatched: create NetBox device with placeholder type 'unknown'. Add to discovery results as unclassified for manual review.
- 7. Timeout per IP: 5 seconds total across all probes. Move on after timeout.

⚠ Scanning a /16 (65,536 IPs) at 5 seconds each sequentially would take 91 hours. Discovery must use `asyncio.gather` with a semaphore limiting concurrent probes to `workers.max_concurrent_polls` (default 50). At 50 concurrent probes, a /16 takes ~110 minutes.

6. Frontend Architecture

6.1 Framework and State

React 18 with hooks. No Redux. State management via React Context for global state (facility info, auth token) and local useState/useReducer for component state. Data fetching via custom hooks wrapping fetch().

6.2 Real-Time Updates

Server-Sent Events (SSE) from GET /api/v1/system/events. The dashboard opens one SSE connection on mount and dispatches events to relevant components via React Context. SSE is simpler than WebSocket for this use case (server-to-client only, auto-reconnect built into EventSource API).

Event routing in React:

- device_discovered → Discovery view updates device count
- poll_result → Device detail view updates register values
- test_started / test_completed / test_aborted → Test execution view updates status
- punch_item → Punch list view adds new item
- manual_confirmation_needed → Modal popup with confirm/deny buttons
- worker_health → System health indicator updates

6.3 Authentication

Single API key configured in platform.yml. Frontend sends it as X-API-Key header on every request. The API key is stored in localStorage after first entry. This is adequate for an on-prem tool used by the commissioning team during a project. No user accounts, no RBAC, no OAuth.

⚠ This is intentionally simple. The platform runs on-prem inside the customer's facility network. If the network is compromised, the API key is the least of the problems. RBAC and multi-user auth are post-v1.

6.4 Page Structure

Route	Component	Data Sources
/	FacilityOverview	GET /punchlist/summary + GET /system/health + SSE
/discovery	DiscoveryView	GET /discovery/status + GET /discovery/results + SSE
/devices	DeviceList	GET /devices + SSE

/devices/:id	DeviceDetail	GET /devices/:id + GET /devices/:id/history + SSE
/tests	TestExecution	GET /tests/history + SSE (live test data)
/tests/:id	TestDetail	GET /tests/status/:id + GET /tests/live/:id (SSE)
/punchlist	PunchList	GET /punchlist + SSE
/checklist	PhysicalChecklist	GET /checklist/:id + POST /checklist/:id/:item_id
/attestation	AttestationViewer	GET /attestation/chain + GET /attestation/verify
/reports	Reports	POST /reports/generate + GET /reports/:id
/import	ImportPage	POST /bim/import + POST /config/generate

6.5 Offline Checklist (Module 19)

The physical verification checklist must work when the technician walks into a mechanical room with no WiFi. Implementation:

- Service worker caches the checklist page and device list at initial load.
- Checklist submissions are stored in IndexedDB when offline.
- When connectivity returns, service worker syncs queued submissions to the API.
- Conflict resolution: if the same checklist item was submitted twice (offline + someone else online), keep the one with the earlier timestamp. Log the conflict.
- Photos are compressed to 1024px max dimension and JPEG quality 80 before upload to minimize sync bandwidth.

7. Physical Checklist Definition

Checklist items are defined per device type as a JSON array in the Config Context alongside registers and active_tests. The schema:

Field	Type	Description
checklist[].id	string	Unique item identifier
checklist[].description	string	What to verify (e.g. 'Verify all mounting bolts torqued to spec')
checklist[].category	string	structural cosmetic safety labeling
checklist[].requires_photo	bool	Whether photo evidence is mandatory for this item
checklist[].acceptance_criteria	string	What constitutes a pass (e.g. 'All bolts torqued to 45 ft-lbs')

Example for a rack-mounted PDU:

- id: 'mounting_bolts', description: 'Verify all rack mounting bolts torqued to spec', category: structural, requires_photo: false
- id: 'cable_labeling', description: 'Verify all power cables labeled at both ends matching cable schedule', category: labeling, requires_photo: true
- id: 'led_status', description: 'Verify all status LEDs show normal (green)', category: cosmetic, requires_photo: true
- id: 'clearance', description: 'Verify minimum clearance maintained per design', category: safety, requires_photo: false

Photo storage: photos are stored as MinIO objects (NOT in WORM bucket) at key: photos/{facility}/{device_id}/{checklist_item_id}/{timestamp}.jpg. The checklist submission itself (completion status, notes, photo reference, attestation hash) goes through the attestation engine into WORM.

8. Security

Concern	Decision	Rationale
API authentication	Single API key in X-API-Key header	On-prem tool, small team, short project. RBAC is post-v1.
API key storage (frontend)	localStorage after first entry	Adequate for on-prem. Not a SaaS product.
NetBox API token	In platform.yml (file permissions 600)	Container-local file. Not exposed to network.
MinIO credentials	In platform.yml + Docker env vars	Standard Docker secret management.
InfluxDB credentials	In platform.yml + Docker env vars	Same pattern.
External API tokens (Genetec, etc.)	In platform.yml	Same pattern.
Attestation key management	No signing keys. Hash-only. The chain integrity comes from WORM immutability, not cryptographic signatures.	Signing keys add complexity without value when the storage is already immutable.
Merkle tree scope	One chain per facility	Simplest model. Multi-chain adds complexity with no benefit for single-facility deployments.
Network exposure	Platform container on facility LAN. No internet exposure.	On-prem. Firewall rules are the customer's responsibility.
Device write commands	identity_register verification before any write. manual_confirmation for dangerous tests.	Prevents writing to wrong device via gateway misroute.
CORS	Allowed origins from platform.yml. Default: http://localhost:8080	Frontend served from same origin in production.

9. Deployment Requirements

Requirement	Specification
OS	Any Linux with Docker Engine 24+. Tested on Ubuntu 22.04/24.04.
Docker Compose	v2.20+
CPU	4 cores minimum. 8 cores recommended for >500 devices.
RAM	16 GB minimum. 32 GB recommended for >500 devices.
Storage	500 GB SSD minimum. WORM data grows at ~1 GB/day per 1000 devices at 30s poll.
Network	1 Gbps Ethernet to facility network. Must be on same subnet as devices or have routing to device subnets.
Ports required outbound	502 (Modbus), 47808 (BACnet), 161 (SNMP), 443 (Redfish/REST APIs)
Ports required inbound	8080 (platform UI). Only accessible from facility LAN.
NTP	Platform server must be NTP-synced. All attestation timestamps depend on accurate clock.
DNS	Platform must resolve NetBox, InfluxDB, MinIO container names. Docker internal DNS handles this.
Internet	Not required during commissioning. Required only for Module 17 (PDF pipeline calls Anthropic API).

9.1 QR Code Format

Module 19 (physical checklist) scans QR codes on equipment to identify the NetBox device. QR code format:

NETBOX:{device_id}

Example: NETBOX:dev-1423

QR codes are generated by NetBox and printed as labels during the BIM import or discovery phase. The format is a simple string prefix. The checklist app strips the NETBOX: prefix and calls GET /api/v1/devices/{device_id}.

10. Logging

Structured JSON logging to stdout. Docker collects via json-file driver. Every log line includes:

Field	Type	Description
timestamp	ISO 8601	Platform NTP time
level	string	DEBUG INFO WARNING ERROR
module	string	Module name (e.g. modbus_worker, test_engine, attestation)
device_id	string null	If related to a specific device
test_id	string null	If related to a specific test
message	string	Human-readable description
data	dict null	Structured event data (register values, error details)

Log levels by module:

- Protocol workers: INFO on poll cycle start/end, WARNING on timeout/retry, ERROR on 3 consecutive failures
- Test engine: INFO on state transitions, WARNING on precondition failures, ERROR on abort/restore_failure
- Attestation: INFO on WORM write, WARNING on WAL fallback, ERROR on chain break
- Discovery: INFO on device found/classified, WARNING on unclassified device
- API: INFO on requests, WARNING on auth failures

11. Updated Module Build Order

With all interface contracts defined, each module now has exact input/output types, exact API endpoints, and exact data structures. No ambiguity.

#	Module	Lang	Lines	Input Types	Output Types
0	Docker Compose + platform.yml	YAML	~100	n/a	Running containers
1	Config Context Loader	Python	~50	JSON file + slug	Config in NetBox
2	NetBox Device Reader	Python	~50	protocol str	list[DeviceInfo]
3	Modbus TCP Poller	Python	~100	DeviceInfo	PollResult
4	InfluxDB Writer	Python	~50	PollResult	InfluxDB point
5	Attestation Engine	Python	~150	AttestationRecord via queue	AttestedRecord in MinIO
6	Modbus Worker	Python	~150	DeviceInfo list	PollResult → Mod 4+5
7	BACnet/IP Poller	Python	~100	DeviceInfo	PollResult
8	SNMP Poller	Python	~100	DeviceInfo	PollResult
9	NVML/DCGM Poller	Python	~100	DeviceInfo	PollResult
10	Active Test Engine	Python	~250	TestRequest via queue	TestResult
11	Discovery Scanner	Python	~200	CIDR list	list[DeviceInfo]
12	REST API Connectors	Python	~400	API config	AttestationRecord via queue
13	Orchestrator	Python	~200	TestRequest via queue	Dispatches to Mod 10
14	Reconciliation Engine	Python	~200	NetBox state	list[PunchListItem]
15	Report Generator	Python	~300	list[PunchListItem] + attestation	PDF + JSON
16	BIM/IFC Import	Python	~200	IFC file	NetBox devices + connections
17	PDF Pipeline	Python	~300	PDF file + slug	Config Context JSON
1	Dashboard	React	~15	API endpoints 4.1-4.10	UI views

8		t	00		
19	Physical Checklist	React	~500	API 4.6 + IndexedDB	Checklist submissions
20	Attestation Viewer	React	~500	API 4.5	Chain verification UI

Additional module not in v5:

Module 21: FastAPI Server

- **What:** The API server implementing all endpoints from Section 4. Routes requests to the appropriate module. Serves frontend static files. Manages SSE event stream.
- **Code:** ~400 lines
- **Test:** Start server with mock modules. Hit every endpoint. Verify request/response matches Section 4 contracts. Test SSE event stream. Test CORS. Test auth rejection on missing/wrong API key.
- **Depends:** All backend modules (imports them as functions)

Module 22: Digital Twin Simulator

- **What:** PyModbus TCP server with registers updated dynamically from OpenModelica physics models. Writable registers trigger model state changes (e.g. write battery_test → UPS model transitions to battery mode → voltage registers follow transient curve).
- **Code:** ~300 lines of glue. Physics models from Modelica Standard Library.
- **Test:** Start simulator with UPS profile. Write battery_test=1 via Modbus. Read voltage registers at 100ms. Verify realistic transient curve (dip, recover, stabilize). Write battery_test=0. Verify return to normal.
- **Depends:** PyModbus, OpenModelica

Grand total: ~7,300 lines Python + React + JSON configs across 23 modules.

12. Time Handling

All timestamps throughout the platform are UTC. No local time anywhere. The platform server must be NTP-synced before commissioning begins.

- Platform timestamps: Python `time.time_ns()` returns UTC nanoseconds. Used for all attestation records.
- InfluxDB timestamps: nanosecond precision UTC. Same as platform.
- MinIO object keys include date in UTC.
- API responses: ISO 8601 with Z suffix (e.g. 2026-04-12T14:30:00.000Z).
- Frontend displays: browser converts UTC to local time for display only. Never sends local time to backend.
- Device timestamps (if any): recorded as `device_timestamp` alongside platform timestamp. Clock skew = `platform_timestamp - device_timestamp`. Logged for audit trail but attestation always uses platform time.

13. Multi-Facility Support

v1 is single-facility. One platform instance per facility. If commissioning multiple facilities simultaneously, deploy separate Docker stacks per facility.

Multi-facility is a v2 feature. When implemented: facility becomes a top-level namespace in NetBox (sites), InfluxDB (tags), and MinIO (key prefix). The orchestrator, attestation chain, and API are all scoped by facility. One platform instance serves multiple facilities.

For now: one Docker Compose stack = one facility. Simple, no namespace complexity, no cross-facility bugs.